

CAPS COMMITTEE TECH DEVELOPER POSITION

TASK - EMAIL DELIVERY SYSTEM

Tech Stack & Write-up

For this task I built a single-file registration desk system using plain HTML, CSS, and JavaScript, no frameworks, no backend. The deliberate choice here was simplicity: the system needs to run on any laptop at an event desk with zero setup. A volunteer just opens one file in Chrome and it works.

For email delivery I used EmailJS, a browser-based email API that lets you send transactional emails directly from JavaScript without needing a server. The way it works is straightforward — when a student is registered, the JS calls the EmailJS API with the student's name and email as variables, which get injected into a pre-built template and sent instantly to their inbox.

The UI was built with a two-column layout — registration form on the left, live log on the right — so the volunteer has a clear action area and can visually confirm each registration without switching screens. I kept the color language simple: green for success, amber for duplicates, red for errors.

Challenges faced

The biggest challenge was getting the email delivery to actually work. I was initially confused about the three separate credentials EmailJS requires — the Public Key, Service ID, and Template ID — and kept mixing them up, which meant emails were silently failing in demo mode. Once I understood what each key represented and where to find them in the dashboard, it clicked. The other learning curve was automating the process end-to-end; I'm comfortable crafting email templates and building layouts with CSS, but wiring the form submission to trigger an API call and handling edge cases like duplicates required careful thought.

How I used AI

I used Claude to scaffold the initial HTML structure and EmailJS integration pattern, which saved significant time on boilerplate. Where I had to step in myself was debugging the API key configuration - the AI generated code that checked whether the key matched a placeholder string, which broke silently when I put my real key in. I had to read through the logic, identify the bug on line 55, and fix it myself. The write-up, design decisions, and understanding of why each piece works the way it does are my own.